Listen

Learning Doc 10: Stage 1, the Foundations in Code
The plumbing every later part of the agent stands on, taught from zero

Implementation, stage 1 of 6

21 October 2026, revised 29 October 2026

Abstract

Docs 01–09 were design. This is the first document about real code: the agent's foundations, built in stage 1. It is written to be read by someone who has never studied these ideas before. Every concept starts with the problem it solves, then the idea in plain words, an everyday analogy (and where that analogy stops being true), a step-by-step example, the real code, the mistakes people usually make, and a short question to check yourself. The answers are at the end. Every program output shown here is copied from a real run on the development laptop.

Contents

1 Before you start **3**

2 What stage 1 built **3**

3 Installing your own package **4**
3.1 The command, piece by piece . . . 4
3.2 The format of `pyproject.toml` . . . 4
3.3 Editable or copied . . . 5
3.4 What `listen.exe` is . . . 5
3.5 PATH and activation . . . 6

4 The command line **6**
4.1 Exit codes: who decided what . . . 7

5 Settings: trust a file only after checking it **7**

6 Dataclasses **8**
6.1 The problem . . . 8
6.2 The idea: let Python write them . . . 8
6.3 Option 1: `frozen=True`, read-only . . . 8
6.4 Option 2: `default_factory`, a fresh default each time . . . 9
6.5 Option 3: `kw_only=True`, solving an ordering rule . . . 9

7 Threads, queues and the worker loop **9**
7.1 Two prerequisites . . . 9
7.2 The loop, line by line . . . 10
7.3 Example: the same pattern with only the standard library . . . 11
7.4 How a stage 2 module will use it . . . 11

8 Errors inside workers **11**
8.1 `Exception` versus `BaseException` . . . 11
8.2 Never put words you heard into an exception message . . . 11
8.3 The error storm, and why errors are still published . . . 12

9 Method resolution order (MRO) **12**
9.1 Prerequisite: inheritance . . . 12
9.2 How the event bus uses the MRO . . . 13
9.3 One copy, never two . . . 13

10 Race conditions and locks **14**
10.1 Two prerequisites . . . 14
10.2 The race, step by step . . . 14
10.3 The lock . . . 14
10.4 Keep the critical section short . . . 15

11 Deadlock **15**
11.1 What it is . . . 15
11.2 Breaking the circle . . . 16
11.3 How Listen avoids it . . . 16

12 The supervisor **16**

13 Dependency injection and fakes **17**
13.1 The problem . . . 17
13.2 The idea: hand it in . . . 17
13.3 Fakes . . . 17
13.4 What to inject, and what not . . . 18

14 pytest: tools and options **18**

15 Logging **19**

16 ruff and CI **19**

17 Key takeaways and mental models **20**

18 Glossary **20**

19 Practice questions **21**

20 Answers **22**

1 Before you start

You need these ideas. Each gets a few sentences here; the sections that use them go deeper.

A program and a process
A *program* is files on disk. A *process* is a program that is running: Windows gave it memory and is executing it. `listen run` starts one process.

The terminal
A window where you type commands (PowerShell or Git Bash). A command is a program name followed by *arguments*: in `listen run`, `run` is the argument.

A Python module and package
A module is one `.py` file. A package is a folder of modules with an `__init__.py` file in it. `listen_agent` is a package; `listen_agent/core/bus.py` is a module inside it.

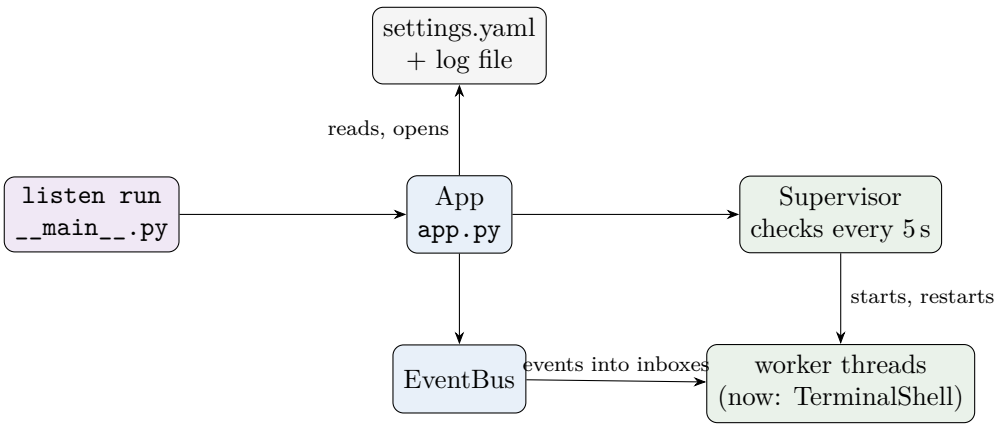
Function, parameter, default
`def f(x=5):` defines a function with a parameter `x` whose *default* is 5 when the caller gives nothing.

Class and object
A class is a blueprint ("what a Book is"); an object is one thing built from it ("this particular book"). `self` inside a method means "the object this method was called on".

Exception
Python's way of saying "something went wrong": it *raises* an exception object, and code can *catch* it with `try` / `except` and carry on.

Earlier docs
Virtual environments and `pyproject.toml` (Doc 02), threads, the event bus and the supervisor as designs (Doc 04), logs, tests and CI (Doc 09).

2 What stage 1 built



What happens when you type `listen run`:

1. The log file `agent/logs/listen.log` is opened.
2. `agent/settings.yaml` is read and checked; a bad value stops here (exit code 2).
3. The `App` creates the event bus, the workers (in stage 1 only the terminal shell) and the supervisor, and starts one thread per worker plus one for the supervisor.
4. "Listen is ready." is printed and logged; the main thread waits.
5. Ctrl+C: "Stopping…", every thread is told to stop, "stopped" is logged, exit code 0.

Why it is done this way

All eleven modules of stages 2–4 need settings, logging, a thread, an inbox and a way to talk to each other. Building that once, first, and testing it alone means each later module only writes its own job. It is also pure Python with no microphone or AI model: the gentlest place to learn the tools.

3 Installing your own package

3.1 The command, piece by piece

```
agent/.venv/Scripts/python -m pip install -e "agent[dev]"
```

Part	Meaning
`agent/.venv/`	the agent's private Python set-up (made by `python -m venv agent/.venv`)
`Scripts/python`	that set-up's own Python (on Linux and Mac the folder is `bin/`)
`-m pip`	"run the module `pip`": Python's package installer, the one belonging to *this* Python
`install`	pip's subcommand (others: `uninstall`, `list`, `show`)
`-e`	**editable**: link to your folder instead of copying it (Section 3.3)
`agent`	the folder holding `pyproject.toml`
`[dev]`	also install the optional group named `dev` (pytest and ruff)
`" "`	quotes stop the terminal treating `[ ]` as a file-name pattern

Watch out

Spaces matter: `python-m` or `install-e` fail. And plain `pip install ...` runs whichever `pip` Windows finds first, which may belong to the system Python, so the libraries land in the wrong place. `agent/.venv/Scripts/python -m pip` cannot be ambiguous.

3.2 The format of `pyproject.toml`

There are two layers of rules. **TOML** is the syntax: `[name]` starts a section (a *table*), `key = value` lines go inside, strings are quoted, lists use `[ ]`, `{ }` is a small inline table, `#` starts a comment, and a dotted name like `[project.scripts]` means "the `scripts` part inside `project`". **Python's packaging standards** (documents called PEPs) then fix which section names exist:

```
[build-system]                          # PEP 518: which tool builds the package
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]                               # PEP 621: name, Python version, dependencies
name = "listen-agent"
requires-python = ">=3.11"              # pip refuses an older Python
dependencies = ["pyyaml>=6.0"]          # ALWAYS installed: needed to RUN

[project.optional-dependencies]         # PEP 621: the "extras"
dev = ["pytest>=8.0", "ruff>=0.6"]      # installed only when you ask for [dev]

[project.scripts]
listen = "listen_agent.__main__:main"   # creates .venv/Scripts/listen.exe
```

```
[tool.ruff]                          # [tool.X]: each tool decides its own keys
line-length = 100
```

Where `[dev]` lives: inside `pyproject.toml`, nowhere else. `dev` is a name we chose; Python gives it no meaning. Several groups install together with a comma, `"agent[dev,docs]"`, and a common convention is an `all` group that points back at the package: `all = ["listen-agent[dev,docs]"]`. Without any `[...]`, only the main `dependencies` are installed.

Watch out

Broken TOML stops pip with a parse error. A *misspelt extra* is sneakier: `"agent[dve]"` only prints a warning ("does not provide the extra") and installs everything *except* pytest and ruff; you find out later when `pytest` is missing.

3.3 Editable or copied

A **normal install** copies your code into `.venv/Lib/site-packages/listen_agent/`; edits do nothing until you reinstall. An **editable install** (`-e`) puts a signpost there instead: "the real code is in `Listen/agent/listen_agent`", so edits apply at once.
It also matters for finding files. `paths.py` starts from its own location:

```
AGENT_DIR = Path(__file__).resolve().parents[2]   # two folders above paths.py's folder
```

Install	`paths.py` lives in	so settings are looked for in
`-e` (ours)	`Listen/agent/listen_agent/core/`	`Listen/agent/settings.yaml` ✓
normal	`.venv/Lib/site-packages/listen_agent/core/`	`site-packages/settings.yaml`: missing, so defaults are used silently

We always install with `-e` while developing. When Listen is packaged as a real app (stage 6), settings and logs will move to `%APPDATA%\Listen`, next to the database.

3.4 What `listen.exe` is

`[project.scripts]` makes pip write `.venv/Scripts/listen.exe`. Opened up, the real file (108,386 bytes) is three parts glued together:

1. a generic **launcher program** (108,032 bytes), the same one inside `pip.exe`, `pytest.exe` and `ruff.exe`;
2. one line naming the Python to use:
   `#!C:\Users\sanid\...\agent\.venv\Scripts\python.exe`;
3. a zipped five-line script:

```
import sys
from listen_agent.__main__ import main
if __name__ == '__main__':
    sys.argv[0] = sys.argv[0].removesuffix('.exe')
    sys.exit(main())
```

So `listen.exe` contains none of your code: it starts *that* Python and calls your `main()`, then passes its exit code back. Because the venv's path is written inside, it always uses the right libraries; because of that same path, moving the venv breaks it (the cure is recreating the venv and reinstalling). Reinstalling overwrites it; while the agent is running, Windows refuses to replace it ("Access is denied"), so stop the agent first.

3.5 PATH and activation

PATH is a list of folders Windows searches, in order, when you type a bare program name; the first match wins. `agent\.venv\Scripts\Activate.ps1` puts the venv's `Scripts` folder at the front of PATH *for that window only*. That is all activation does. A real run in a window that was *not* activated:

```
.\agent\.venv\Scripts\listen.exe --version   ->  listen 0.1.0   (exit code 0)
listen --version                             ->  not recognized (Scripts not on PATH)
```

How you start it	Not activated	Why
`agent\.venv\Scripts\listen.exe run`	works	full path; the exe names its Python itself
`agent/.venv/Scripts/python -m listen_agent run`	works	you chose the venv's Python
`listen run`	fails	a bare name needs `Scripts` on PATH
`python -m listen_agent run`	fails	finds the *system* Python: `No module named 'yaml'`

Concept: pip + venv, uv, uvicorn and curl

uv (by Astral, who also make ruff) replaces pip and venv in one very fast tool and writes a **lock file**, `uv.lock`, recording the exact version of every library. Without one, `ruff>=0.6` means "whatever is newest on the day you install", so CI can turn red after a new release with no code change (the "ruff is not pinned" review finding). Stage 2 will switch to uv. **uvicorn** is unrelated despite the name: it is the web server that will run the FastAPI app in stage 4 (FastAPI is the kitchen, uvicorn the door and waiter). **curl** is a command-line program for sending web requests; in Windows PowerShell type `curl.exe`, because plain `curl` is a different PowerShell command.

4 The command line

```
def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(prog="listen", description="Listen voice agent")
    parser.add_argument("--version", action="version", version=f"%(prog)s {__version__}")
    commands = parser.add_subparsers(dest="command", metavar="command")
    commands.add_parser("run", help="start the agent (Ctrl+C stops it)")
    return parser

def main(argv: list[str] | None = None) -> int:
    parser = build_parser()
    args = parser.parse_args(argv)
    if args.command == "run":
        return run_agent()
    parser.print_help()
    return 0
```

- `build_parser()` only *describes* the command line: which words `listen` accepts. `->` `argparse.ArgumentParser` is a type hint for readers.
- `prog` is the name in messages; `add_subparsers(dest="command")` stores the chosen subcommand in `args.command`; `add_parser("run", ...)` registers `run`.

- `%(prog)s` is a placeholder filled in later by argparse. The line is formatted twice: the `f"..."` fills `{__version__}` now, giving `"%(prog)s 0.1.0"`; argparse fills `%(prog)s` when you type `--version`, giving `listen 0.1.0`.
- `argv=None` means "read what was typed"; tests pass a list instead.
- `if __name__ == "__main__"` (at the end of the file) is true only when the file is run, not when a test imports it.

`--help` and `print_help()` are not the same thing. argparse adds `-h/--help` to every parser by itself; it prints the help and exits *inside* `parse_args`. `print_help()` at the end handles a different case: you typed `listen` alone, so `args.command` is `None`. Without it, `listen` would silently do nothing. Every path, from real runs:

You type	What happens	Exit code
`listen --version`	`parse_args` prints `listen 0.1.0` and exits	0
`listen --help`	`parse_args` prints the help and exits	0
`listen run`	`return run_agent()`	0 / 2 / 3
`listen`	falls through to `print_help()`	0
`listen bogus`	`error: argument command: invalid choice: 'bogus'`	2

4.1 Exit codes: who decided what

Code	Decided by	Meaning
0	every program, every OS	success
1	Python	an uncaught exception crashed the program
2	argparse (an old Unix habit)	wrong usage: bad command or option
2	**us**: `EXIT_BAD_SETTINGS`	fix `settings.yaml`; restarting would not help
3	**us**: `EXIT_FATAL`	crash loop or hang; the launcher (stage 2) restarts

Ours live in `app.py`. Code 2 is used twice, so a launcher could not tell "you typed a bad command" from "fix your settings"; a different number will be chosen before the stage 2 launcher relies on it.

5 Settings: trust a file only after checking it

`load_settings()` reads `settings.yaml` with **PyYAML**, the library that turns YAML text into Python data (Python has built-in JSON and TOML readers, but no YAML one). The real result for our file:

```
{'hotkeys': {'continuous_toggle': 'ctrl+alt+c', 'push_to_talk': 'ctrl+alt+space'},
 'mode': 'wake',
 'thresholds': {'continuous_extra': 0.1, 'hearing': 0.6, 'high_risk_hearing': 0.75,
                'high_risk_match': 0.9, 'match': 0.75}}
```

Then every key and value is checked; a problem stops start-up with a message that starts with the key:

```
Can't start: threshold: unknown setting
Can't start: thresholds.match: must be a number from 0 to 1, got True
Can't start: settings.yaml: can't read it (...); save it as UTF-8
```

Why it is done this way

"Ignore what you don't understand and use defaults" sounds friendly but is the dangerous choice: a typo like `threshold:` would silently do nothing. Fail loudly, early, and say where.

Watch out

Three traps, each with a test. (1) In YAML, unquoted `yes/no` become `True/False`, and in Python `True == 1`, so `bool` is refused explicitly. (2) Notepad can add an invisible byte-order mark; reading with `utf-8-sig` removes it. (3) Notepad can also save as UTF-16 or Windows-1252; those used to crash with a raw `UnicodeDecodeError`, and now give the clear message above with exit code 2. **Always `safe_load`:** plain `yaml.load` can build arbitrary Python objects from a crafted file, even run code; `safe_load` only builds text, numbers, lists and dicts. PyYAML also loses comments when *writing*, so stage 4's web edits to command files will use `ruamel.yaml`, which keeps them.

6 Dataclasses

6.1 The problem

A class that mostly *holds data* needs repetitive code. A book, by hand:

```
class Book:
    def __init__(self, title, pages=100):      # 1. store the values
        self.title = title
        self.pages = pages
    def __repr__(self):                        # 2. how it prints
        return f"Book(title={self.title!r}, pages={self.pages!r})"
    def __eq__(self, other):                   # 3. when two count as equal
        return (self.title, self.pages) == (other.title, other.pages)
```

Nine lines for two fields, and each new field means editing all three methods.

6.2 The idea: let Python write them

```
@dataclass
class Book:
    title: str
    pages: int = 100
```

`@dataclass` is a **decorator**: a function that takes the class you just wrote and returns an improved one. It reads the field list (name, type, default) and writes `__init__`, `__repr__` and `__eq__`. Real output:

```
print(Book("Atoms"))              ->  Book(title='Atoms', pages=100)
Book("Atoms") == Book("Atoms")    ->  True
```

6.3 Option 1: `frozen=True`, read-only

```
f = FrozenBook("Atoms"); f.title = "x"
->  FrozenInstanceError: cannot assign to field 'title'
```

Why it is done this way

One event object is put into several inboxes and read by several threads at once. If any thread could change it, the others would see it change under them. Frozen makes that impossible; the same goes for `Settings`, which must not change after it has been checked.

6.4 Option 2: `default_factory`, a fresh default each time

Writing `at: datetime = datetime.now(UTC)` runs the clock *once*, when the class is defined. Real result: two events made at different moments had the *same* time. The fix hands dataclass a function to call for each new object:

```
at: datetime = field(default_factory=lambda: datetime.now(UTC))   # called per object
```

Rule: if a default must be new for each object (now, an empty list, an empty dict), use `default_factory`. In `events.py` the function is the small named `_now`.

6.5 Option 3: `kw_only=True`, solving an ordering rule

A field without a default may not follow one with a default, because `Bad(5)` would be ambiguous:

```
TypeError: non-default argument 'module' follows default argument
```

That is exactly our case: the parent `Event` has `created_at` (with a default) and each child adds required fields. `kw_only=True` on the parent means "`created_at` only by name", so dataclass moves it to the end:

```
@dataclass(frozen=True, kw_only=True)
class Event:
    created_at: datetime = field(default_factory=_now)

@dataclass(frozen=True)
class ModuleError(Event):
    module: str
    error: str

ModuleError("voice", "no mic")      # created_at filled in automatically
```

Watch out

Use a dataclass when a class is mainly a named bundle of values (settings, events, a row of data). When it is mainly behaviour (a worker, a supervisor), write a normal class.

Quick check 1 (answer in Section 20)

Why would `at: datetime = datetime.now(UTC)` give every event the same time, and what does `default_factory` change?

7 Threads, queues and the worker loop

7.1 Two prerequisites

Concept: thread

One line of work inside a program. Listen runs one per module, so the transcriber can work on a recording while the listener keeps hearing. On a single processor core they take turns, and **Windows decides when to switch**, possibly between any two steps.

Concept: queue

A `queue.Queue` is a thread-safe waiting line: one thread `puts`, another `gets`, in order. `get(timeout=1)` waits at most one second, then raises `queue.Empty`. Every worker owns one: its **inbox**.

7.2 The loop, line by line

```
_NOTHING = object()

def run_once(self) -> None:
    self.beat()
    try:
        message = self.inbox.get(timeout=self.wait_seconds)
    except queue.Empty:
        message = _NOTHING
    try:
        if message is _NOTHING:
            self.idle()
        else:
            self.handle(message)
    except Exception as exc:
        log.exception("%s failed handling %s", self.name, type(message).__name__)
        if not isinstance(message, ModuleError):
            self.bus.publish(ModuleError(self.name, f"{type(exc).__name__}: {exc}"))

def run(self, stop: threading.Event) -> None:
    while not stop.is_set():
        self.run_once()
```

Line	What it does and why
`_NOTHING = object()`	a brand-new object equal to nothing else: a **sentinel** meaning "no message". `None` cannot play this role because `None` can be a real message.
`run_once`	one pass of the loop, so tests can run exactly one pass without threads
`self.beat()`	record the heartbeat first, so even an idle worker beats every pass
`inbox.get(timeout=...)`	take the next message, waiting at most `wait_seconds`. Waiting forever would never notice `stop` and never beat.
`except queue.Empty`	that is how the queue says "nothing came"; store the sentinel
`if message is _NOTHING`	`is` asks "the very same object?", right for a sentinel; `==` asks "equal value?"
`self.idle()`	periodic work, such as "has the default microphone changed?"
`self.handle(message)`	the module's real job, written by each module
`except Exception as exc`	any normal error is caught here, so one bad message cannot kill the thread
`log.exception(...)`	an ERROR line *plus the traceback* in the log; only the message's *type* is logged, never its contents
`if not isinstance(...)`	the brake against the error storm (Section 8.3)
`publish(ModuleError(...))`	tell the rest of Listen, e.g. the terminal shows `error in voice: OSError: no mic`
`while not stop.is_set()`	`stop` is a shared on/off flag (`threading.Event`); setting it ends every loop within about a second

The pattern appears in almost every background worker, in any language: *loop until told to stop: wait for work but not forever; nothing came, do routine checks; work came, do it; it failed, record it, report it, keep going.*

7.3 Example: the same pattern with only the standard library

A `Doubler` worker with the same `run_once`/`run`; its `handle` prints `message * 2`. Messages `4`, `None`, `"ab"`. Real output:

```
double of 4 is 8
error, but still running: TypeError        <- None * 2 fails
double of ab is abab                       <- the next message still works
idle passes: 4 | thread alive: False       <- stopped cleanly when asked
```

7.4 How a stage 2 module will use it

A module inherits the loop and only writes its job:

```
class Feedback(Worker):
    name = "feedback"
    def __init__(self, bus):
        super().__init__(bus)                      # inbox, heartbeat, loop
        bus.subscribe(CommandExecuted, self.inbox)
    def handle(self, message):
        winsound.PlaySound("done.wav", winsound.SND_FILENAME)
```

Daemon threads. Workers start with `daemon=True`: when the main program ends, they end with it, so a truly stuck thread cannot stop Listen from exiting.

Quick check 2 (answer in Section 20)

Why does `get` use a one-second timeout instead of waiting forever for a message? Give two reasons.

8 Errors inside workers

8.1 `Exception` versus `BaseException`

`except Exception` catches ordinary errors (`ValueError`, `OSError` ...) but not `BaseExceptions` such as `SystemExit` and `KeyboardInterrupt`. Those escape, the thread dies (logged with its traceback), and the supervisor replaces it (Section 12). Catching *everything* would also swallow "please exit".

8.2 Never put words you heard into an exception message

Anything in an exception message reaches `listen.log` (through `log.exception`) and the terminal (through `ModuleError`). Logs get attached to bug reports and pasted into GitHub issues, so the privacy rule (spec 9.2) is: no spoken words in logs.

```
raise IntentError(f"no command matches '{text}'")    # BAD: your sentence ends up in the
    log
raise IntentError("no command matches")              # good: what went wrong, not what
    was said
raise SlotError(f"{{app}} slot: unknown app ({len(value)} characters)")   # a size is
    fine
```

Python itself does this, real output:

```
int("open vs code")             ->  invalid literal for int() with base 10: 'open vs
    code'
{}["search what is an atom"]    ->  KeyError: 'search what is an atom'
```

So code that handles your sentences checks before converting, or replaces the message:

```
try:
    number = int(word)
except ValueError:
    raise SlotError("{number} slot: not a number") from None   # drop the original
    message
```

Words *are* allowed on your own screen (the terminal), in the private local database, and in the log only through `words()` with debug on.

8.3 The error storm, and why errors are still published

The terminal shell listens to *every* event, `ModuleError` included. If printing ever fails (console closed), the generic worker rule fires: the shell fails, publishes a `ModuleError`, receives it, fails to print that, publishes another… The review measured about 100,000 a second, enough to rotate away the whole log. Why not just stop publishing errors?

Option	For	Against
Never publish (log only)	simplest; no storm	errors invisible until you open the log; nothing can react (pop-up, red tray icon)
Always publish (original)	everything visible	the storm
Publish, except when failing on a `ModuleError` (chosen)	visible; loop impossible; one line in the shared loop protects every future module	that one failure goes only to the log, which loses nothing: if printing is broken it could not be shown anyway
Rate limit	also stops other floods	more code; still floods at the limit; drops real errors
Fix only the shell	targeted	future modules that listen to errors could repeat the bug

Lesson: anything that both *sends* a kind of message and *receives* that same kind needs a brake. Two e-mail vacation auto-replies answering each other forever is the same bug.

Quick check 3 (answer in Section 20)

A stage 2 matcher fails on the sentence “call mum” and raises `ValueError(f"no slot for {text}")`. Where does “call mum” end up, and how should the line be written?

9 Method resolution order (MRO)

9.1 Prerequisite: inheritance

`class Dog(Animal):` reads “a Dog is an Animal, plus whatever I write here”. The child gets everything the parent has and may **override** (replace) any of it.

```
class Animal:
    def sound(self): return "some noise"
    def legs(self):  return 4
class Dog(Animal):
    def sound(self): return "woof"
class Puppy(Dog):
    pass
```

Step	`p.sound()`: look in	`p.legs()`: look in
1	`Puppy`: no	`Puppy`: no
2	`Dog`: **yes**, `"woof"`, stop	`Dog`: no
3		`Animal`: **yes**, 4, stop

That search order is the **MRO**, and Python stores it: `Puppy.__mro__` is `['Puppy', 'Dog', 'Animal', 'object']` (real output), the family read from the child upwards. `object` is the ancestor of every class.

Analogy

Looking for a hammer: your room first, then your parents' garage, then your grandparents' shed, and you use the first one you find. *Where it breaks:* a person might pick the nicest hammer; Python always takes the *first*, which is why `Dog`'s `"woof"` wins.

Watch out

A child does not copy its parents' methods. Python searches for them on every call, so changing `Animal.legs` later changes it for every `Puppy`.

9.2 How the event bus uses the MRO

A child is also each of its parents: `isinstance(err, Event)` is `True` for a `ModuleError`. So the shell subscribes once, to `Event`, and receives every kind of event, including ones invented later. `publish` walks the event's MRO and, at each class, collects whoever subscribed to that exact class. Shell subscribed to `Event`, an "alerts" module to `ModuleError`; publish a `ModuleError`:

Step	Class	Subscribed	Delivery list
1	`ModuleError`	alerts	[alerts]
2	`Event`	shell	[alerts, shell]
3	`object`	nobody	[alerts, shell]

Real run: `shell got: 1 | alerts got: 1`. Method lookup *stops* at the first match; the bus walks the *whole* list and collects every match. Siblings (two children of `Event`) are never in each other's MRO: a `Transcribed` reaches the shell but not alerts.

9.3 One copy, never two

If one inbox subscribes at two levels (`Event` and `ModuleError`), a naive bus adds it twice. Real run: naive bus 2 copies, real bus 1. One line prevents it:

```
for cls in type(event).__mro__:
    for inbox in self._subscribers.get(cls, ()):
        if inbox not in inboxes:          # already on the list? skip it
            inboxes.append(inbox)
```

Quick check 4 (answer in Section 20)

`Transcribed(Event)` and `HeardInContinuous(Transcribed)`. The shell subscribes to `Event`; module X subscribes to both `Transcribed` and `HeardInContinuous` with one inbox. One `HeardInContinuous` is published. Walk the MRO: how many copies do X and the shell get?

10 Race conditions and locks

10.1 Two prerequisites

Threads take turns and Windows chooses the moment of each switch. And `counter = counter + 1` is three steps, not one: **read** the value, **add** 1, **write** it back. A switch can land between them.

10.2 The race, step by step

Two threads each add 1; `counter` starts at 5; the right answer is 7.

Moment	Thread A	Thread B	counter
1	reads 5		5
2	*(switch)*	reads 5	5
3		adds 1, writes 6	6
4	adds 1 to the 5 it read, writes 6		**6** (wrong)

A's "5" was out of date when it wrote. That is a **race condition**: the result depends on who gets there first. A real run, two threads adding 1,000 each (with a tiny pause between read and write, to make the switch land there often): `expected 2000, got 1000`. A race gives *different* wrong answers on different runs. If A had finished all three steps before B read, B would have read 6 and written 7: the danger is only a switch in the *middle*.

Analogy

Two people check a shared bank account at the same moment (both see Rs 500) and each deposits Rs 100; each phone saves 500 + 100 = 600. One deposit vanished. *Where it breaks:* with threads it is one processor switching, not two people acting at once; the effect is the same.

10.3 The lock

A **lock** is a single key. Only the thread holding it may run the guarded code, called the **critical section**; others wait at the door.

```
lock = threading.Lock()          # ONE key, shared by every thread
with lock:                       # wait for the key, take it
    current = counter            # read
    counter = current + 1        # write
                                 # leaving the block always returns the key, even on an
    error
```

Moment	Thread A	Thread B	Key	counter
1	takes key, reads 5		A	5
2	*(switch)*	wants key, **waits**	A	5
3	writes 6, returns key		nobody	6
4		takes key, reads 6	B	6
5		writes **7**, returns key	nobody	7

Real run with the lock: `expected 2000, got 2000`.

Watch out

(1) Every piece of code that touches the shared thing must use the *same* lock. (2) Creating the lock inside the loop (`with threading.Lock():`) gives every pass its own key, so nobody

ever waits and nothing is protected; `EventBus` makes `self._lock` once, in `__init__`. (3) Do not put more inside the lock than needed (next subsection).

10.4 Keep the critical section short

One family phone book, three people calling Mum: looking up the number takes seconds, the call takes minutes. Holding the book for the whole call makes the others queue for nothing; copying the number, putting the book back, then calling lets everyone call at once. The same in code, three threads, each copying from a shared dict and then doing 0.5 s of slow work. Real run:

```
call_holding_the_notebook    3 people done in 1.50 s    <- slow work inside the lock
call_after_copying           3 people done in 0.50 s    <- slow work after releasing it
```

`EventBus.publish` follows the second way: under the lock it only builds the private list of inboxes, then releases the key and delivers.

Quick check 5 (answer in Section 20)

In the phone-book run, why does the third thread finish at 1.5 s when the slow work is inside the lock? Say what each thread is doing during the first 0.5 s.

11 Deadlock

11.1 What it is

Two threads each **hold** something the other needs and each **waits** for the other to let go. Neither does, so both wait forever: no crash, no error, just frozen. Ravi takes the pen and wants the paper; Meena takes the paper and wants the pen:

Moment	Ravi	Meena	Pen	Paper
1	takes pen		Ravi	–
2		takes paper	Ravi	Meena
3	wants paper, waits		Ravi	Meena
4		wants pen, waits	Ravi	Meena
5...	waiting	waiting	Ravi	Meena

A real run, with a 2-second limit on each wait so the demo ends:

```
Ravi:  got the pen, now waiting for the paper...
Meena: got the paper, now waiting for the pen...
Ravi:  still waiting after 2 s -> STUCK
Meena: got the pen, writing
```

Ravi gave up, which put the pen down, and only then could Meena finish: the timeout *broke* the deadlock. Without it, the program would sit there forever, silently.

The four ingredients must all be present: (1) things only one thread can hold; (2) a thread keeps what it holds while waiting for more; (3) nobody can take a thing away; (4) a **circle** of waiting. Remove any one and a deadlock is impossible.

Analogy

Two cars nose to nose on a one-lane bridge. *Where it breaks:* drivers can see each other and agree that one reverses; threads cannot see the circle at all, which is why deadlocks look like quiet, not like errors.

11.2 Breaking the circle

If everyone takes the pen first, then the paper, the second person waits *empty-handed* and blocks nobody. Real run: `both finished, no timeout needed`.

11.3 How Listen avoids it

Ingredient	Escape route	In Listen
circle of waiting	take locks in one agreed order	no code holds two locks at once
hold while waiting	never hold a lock while waiting on something else	the bus copies, releases, then delivers
nothing taken away	timeouts: give up, release, retry	`get(timeout=1)`, `join(timeout=2)`
all else fails	detect and restart	heartbeats, supervisor, launcher

Why the bus delivers outside its lock. Imagine inboxes with a size limit and a *bad* `publish` that calls `inbox.put(event)` while holding the bus lock. The voice thread holds the lock and waits for space in the shell's full inbox; the shell would make space, but first wants the bus lock to publish its own event. That is the pen and the paper in disguise. Releasing the lock before `put` means the voice thread waits holding nothing. Today inboxes have no limit, but the bus is written so adding one later cannot introduce this.

Quick check 6 (answer in Section 20)

If Ravi and Meena both always take the pen first, can they still deadlock? Which ingredient does that rule remove?

12 The supervisor

Every 5 seconds the supervisor looks at each worker:

```
if not self.threads[worker.name].is_alive():          # dead
    if not self._may_restart(worker.name, now):
        self.on_fatal(f"{worker.name} keeps crashing")  # 4th death in a minute
        return
    self._start_thread(worker)                          # new thread, same worker + inbox
elif now - worker.last_beat > HUNG_AFTER:              # alive but silent for 30 s
    self.on_fatal(f"{worker.name} stopped responding")
```

- A Python thread cannot be restarted, so a dead worker gets a *new* thread running the *same* worker; the inbox belongs to the worker, so waiting messages survive.
- Recent restart times are kept in a `deque` (a list cheap to trim from the front); times older than 60 s are dropped; a fourth death within the minute is a crash loop.
- Python cannot safely kill a hung thread, so the cure is restarting the whole agent: `on_fatal` sets exit code 3 and the launcher takes over.

Why 30 seconds and not 5? A heartbeat is only recorded *between* jobs. Some normal jobs take seconds (Whisper transcribing, launching a heavy app, Windows briefly pausing Python). A 5-second limit would restart Listen in the middle of transcribing your sentence: the command is lost, the models reload, and one of the launcher's three restarts per 10 minutes is used up. Waiting longer for a rare real hang costs little. Rule of thumb: a timeout sits well above the longest normal case.

Sleep is not a hang. On Windows, `time.monotonic` keeps counting while the laptop sleeps, so after an hour with the lid closed every heartbeat looks an hour old. The supervisor checks every 5 s, so a gap of more than 30 s between two of *its own* checks means the whole laptop slept; it logs "clock jumped" and refreshes every heartbeat. A really stuck worker is still caught 30 s later. This is why the hang tests advance the fake clock in 5 s steps with a check after each: a single 31 s jump looks exactly like sleep.

13 Dependency injection and fakes

13.1 The problem

A **dependency** is anything outside your code that it relies on: the clock, the screen, apps, files. Code that fetches them itself cannot be tested in a controlled way:

```
def greeting_v1():
    hour = datetime.now().hour          # grabs the REAL clock itself
    return "Good morning" if hour < 12 else "Good evening"
```

"At 9 am it says Good morning" can only be tested in the morning.

13.2 The idea: hand it in

```
def greeting_v2(now=datetime.now):      # the clock is a parameter; real clock by default
    hour = now().hour
    return "Good morning" if hour < 12 else "Good evening"
```

Call	`now().hour`	Result (real run)
`greeting_v2()`	real hour	`Good evening`
`greeting_v2(lambda: datetime(2026,10,27,9,0))`	9	`Good morning`
`greeting_v2(lambda: datetime(2026,10,27,21,0))`	21	`Good evening`

That is **dependency injection**: the dependency is passed in instead of grabbed. (`lambda: X` is a one-line function that returns X.)

Analogy

A chef who cooks with whatever tomatoes you put on the counter: market tomatoes on a normal day, practice ones during a test. *Where it breaks:* a chef notices plastic tomatoes; code does not. It uses anything that *behaves* correctly, which is what makes fakes possible.

Watch out

`def greeting_v2(now=datetime.now()):`, *with* brackets, calls the clock once when the function is defined and freezes that moment as the default: the same trap as `default_factory` in Section 6.

13.3 Fakes

A **fake** is a simple stand-in that accepts the same calls. The shell only ever does `out("text")`, so `lines.append` works just as well as `print`, and the test reads the list. Listen's main fake is the clock:

```
class FakeClock:
    def __init__(self, start=1000.0): self.now = start
    def __call__(self): return self.now          # clock() works, like time.monotonic()
```

```
    def advance(self, seconds): self.now += seconds   # time moves only when the test
    says
```

The three hang and sleep tests cover more than two hours of pretend time; real run: `3 passed in 0.10s`.

Name	What it is	Example
Real	the actual thing	`time.monotonic`, `print`
Fake	a working, simplified version	`FakeClock`; `lines.append`
Stub	returns a fixed answer, nothing more	`lambda: datetime(2026,10,27,9,0)`
Mock	a stand-in that records how it was called	a recorder checked with "was it called with X?"

Watch out

Share one fake wherever values are compared. If the supervisor got a `FakeClock` but the worker kept the real clock, the subtraction mixes two clocks. Real numbers: worker's heartbeat 1,009,750.9, supervisor's "now" 1,031.0, silence −1,008,719.9 seconds: the hang is never detected. Like two friends comparing an Indian watch with a New York phone: each clock is fine, comparing them is the bug. Also: keep fakes tiny so they cannot drift from reality, and assert on *your* code's behaviour, not on the fake's.

13.4 What to inject, and what not

Inject what is **slow, uncontrollable, or has real-world side effects**: the clock, the screen, apps, volume, the microphone. Do not inject `len()`, `sorted()` or our own pure functions such as the stage 2 matcher: test those directly with inputs and outputs. In stage 2 the executor will take an `actions` object; tests pass a `FakeActions` that records "`launch_app('Visual Studio Code')`" in a list instead of opening anything.

Quick check 7 (answer in Section 20)

In `greeting_v2(now=datetime.now)`, why must there be no brackets after `datetime.now`?

14 pytest: tools and options

Tool	What it does, and where we used it
fixture	prepares something a test needs; a test asks for it by naming it as a parameter (`bus`, `clock`, `stop`)
`yield` in a fixture	code after `yield` is clean-up (remove a log handler, set `stop`)
`conftest.py`	fixtures shared by every test in the folder; the fake clock lives there
`tmp_path`	a fresh empty folder per test
`capsys` / `caplog`	capture printed text / log records
`monkeypatch`	replace something temporarily; undone after the test
`parametrize`	one test function, many inputs (8 bad settings files)
`pytest.raises`	passes only if the expected error is raised

Option	Stands for	Effect
`-v`	`--verbose`	one line per test instead of dots (`-vv`: more detail on failures)
`-k word`	keyword	only tests whose name matches; understands `and`, `or`, `not`
`-x`	`--exitfirst`	stop at the first failure
`--lf`	`--last-failed`	re-run last time's failures (remembered in `.pytest_cache/`)
`--ff`	`--failed-first`	last failures first, then the rest
`file.py::name`	a *node id*	one test by its full address

These are pytest's own options: they work for every test file, including the server's in stage 4. One dash with a letter is short (`-vx` combines two); two dashes with a word is long and readable. Every task began with a test that failed for the right reason (`ModuleNotFoundError`); a test that passes before the code exists tests nothing.

15 Logging

```
LOG_FORMAT = "%(asctime)s %(levelname)-5s %(name)-8s %(message)s"
handler = RotatingFileHandler(log_dir / "listen.log", maxBytes=1_048_576, backupCount=5,
                              encoding="utf-8")
```

A **logger** is what code writes to (`logging.getLogger("core")`); a **handler** decides where lines go; a **formatter** how they look (`-5s` pads to 5 characters). At 1 MB, `listen.log` becomes `listen.log.1` and so on up to `.5`. A real line:

```
2026-10-21 15:09:37,732 INFO  core     ready (version 0.1.0, mode wake)
```

Anything you said is wrapped before logging:

```
log.info("heard %s", words(text))     # "[words hidden]" unless debug is on
```

The debug flag is a plain module variable that starts `False` in every new process, which is exactly "debug lasts until restart".

16 ruff and CI

ruff reads code without running it and reports likely mistakes; `--fix` repairs the safe ones. Families switched on: **E** style, **F** real mistakes (unused imports, undefined names), **I** import order, **B** likely bugs (e.g. a list as a default argument), **UP** old syntax a newer Python writes better.

	PyCharm inspections	ruff
Runs	only in the IDE	terminal, CI, any editor
Settings	your IDE (not in the repo)	`pyproject.toml` (committed)
Ignorable?	yes, a squiggle	not in CI: a problem turns the check red

CI cannot run PyCharm, so ruff is the rule the project enforces (PyCharm's Ruff plugin shows its warnings while you type). `.github/workflows/ci.yml` runs `ruff check agent` and `pytest agent` on a fresh Windows machine on every push; the first run passed all 71 tests. **Pinned** means one exact version (`ruff==0.17.1`); ours are ranges (`ruff>=0.6`), so a new release can add a check and turn CI red with no code change. A lock file fixes that.

17 Key takeaways and mental models

Idea	Mental model
Editable install	a signpost to your folder, not a copy
`listen.exe`	"start that Python, call `main()`", with nothing of yours inside
Activation	only puts `Scripts` at the front of PATH, for one window
Settings	trust a file only after checking it; fail loudly, naming the key
Dataclass	a named bundle of values whose boring methods are written for you
Worker loop	wait (not forever), do the job, report failures, keep going
MRO	a list of "who I am", most specific first; methods use the first match, the bus uses all
Race condition	a switch landing in the middle of read-add-write
Lock	the one key to a small room; do the minimum inside, then hand it back
Deadlock	each holding half of what both need; never wait while holding
Dependency injection	tools are put on the code's counter; tests put toy ones there
Fake	a tiny stand-in that behaves the same for the calls that matter

18 Glossary

argparse
Python's standard library for reading command-line arguments.

Critical section
Code only one thread may run at a time, guarded by a lock.

Daemon thread
A thread that ends automatically when the main program ends.

Dataclass
A class whose `__init__`, equality and printing are generated from its fields.

Deadlock
Threads each holding something another needs, all waiting forever.

Decorator
A function that takes a function or class and returns an improved one (`@dataclass`).

Dependency
Something outside your code that it relies on: clock, screen, files, apps.

Dependency injection
Passing a dependency in as a parameter instead of fetching it inside.

Editable install
`pip install -e`: the installed package points at your source folder.

Exit code
The number a program ends with; 0 is success.

Extra
An optional group of dependencies, e.g. `[dev]`.

Fake A working, simplified stand-in used by tests.

Fixture
A pytest helper that prepares (and cleans up) something tests need.

Frozen
A dataclass whose fields cannot be changed after creation.

Handler / formatter
Where log lines go / how they look.

Heartbeat
A timestamp a worker updates on every loop pass, proving it is alive.

Inheritance
Building a class on another: the child gets everything the parent has.

Lock A single shared key that lets one thread at a time into a critical section.

Lock file
A record of the exact version of every installed library.

Mock
A stand-in that records how it was called.

Monotonic clock
A clock that only moves forward; right for measuring durations.

MRO
Method resolution order: a class's family list, from itself up to `object`.

Override
A child replacing a method it inherited.

PATH
The ordered list of folders Windows searches for a bare program name.

Pinned
Fixed to one exact version.

Queue
A thread-safe waiting line; each worker's inbox.

Race condition
A bug that depends on which thread gets somewhere first.

Sentinel
A unique object used as a marker, e.g. "nothing arrived".

Stub A stand-in that only returns a fixed answer.

Subcommand
One action of a multi-action program: `listen run`.

Thread
One line of work inside a program; threads take turns.

TOML
The configuration syntax of `pyproject.toml`.

19 Practice questions

Check yourself

1. *(easy)* You change `settings.yaml` to `thresholds: { match: 80 }`. What does `listen run` print, and what is the exit code?

2. *(easy)* Why does `python -m listen_agent run` fail in a window where the venv is not activated, while `agent\.venv\Scripts\listen.exe run` works?
3. *(easy)* What is `Dog.__mro__`, written as names?
4. *(medium)* A worker's `handle` raises `ValueError` on every message. Does the supervisor restart its thread? Why or why not?
5. *(medium)* Two threads only *read* `Settings` after start-up. Do they need a lock?
6. *(medium)* `test_app.py` uses `out=lines.append` but the real clock. Why is the real clock fine there but not in the supervisor tests?
7. *(harder)* Rewrite `publish` badly, with `inbox.put(event)` inside the `with self._lock:` block, and describe step by step a situation in which Listen freezes.
8. *(harder)* Stage 2's feedback module plays chimes with `winsound`. Sketch a design in which a test can check "the done chime was played after a command ran" without any sound.

20 Answers

Quick checks

1. The default expression runs once, when the class is defined, and that single moment is stored and shared. `default_factory` stores a *function* and calls it for each new object.

2. A worker waiting forever would never see `stop` being set, so shutdown would hang; and it would never record a heartbeat, so the supervisor would think it hung.

3. Into the exception message, so into `listen.log` through `log.exception` and onto the terminal through `ModuleError`. Write `ValueError("no slot matched")`: what went wrong, not what was said.

4. MRO: `HeardInContinuous`, `Transcribed`, `Event`, `object`. Step 1 adds X; step 2 finds X again and skips it; step 3 adds the shell. X gets 1 copy, the shell 1 copy.

5. From 0 to 0.5 s thread 1 holds the lock and does its slow work; threads 2 and 3 only wait for the key. Thread 2 works from 0.5 to 1.0 s and thread 3 from 1.0 to 1.5 s: single file, though none of them needed the lock for the slow part.

6. No. Whoever gets the pen second waits holding nothing, so nobody blocks them. The rule removes the circular wait.

7. Without brackets the parameter holds the clock *function*, called afresh on each use. With brackets the clock is read once, when the function is defined, and that frozen moment becomes the default.

Practice questions

1. `Can't start: thresholds.match: must be a number from 0 to 1, got 80`; exit code 2.

2. A bare `python` finds the system Python, which has neither PyYAML nor Listen; `listen.exe` has the venv's Python path written inside it.

3. `Dog`, `Animal`, `object`.

4. No. The error is caught inside the loop and published as `ModuleError`; the thread stays alive and keeps beating. Only a thread that *dies* is replaced.

5. No. A race needs someone *writing*; `Settings` is frozen, so nobody can.

6. `test_app.py` only waits up to two real seconds for a printed line; nothing in it depends on 30 s passing. The supervisor tests need to simulate minutes and hours.

7. The voice thread takes the bus lock and calls `put` on the shell's full inbox, waiting for space; the shell, to make space, first tries to publish and waits for the bus lock. Each waits for the other: deadlock.

8. Give the module a `player` parameter defaulting to a function that calls `winsound`; the test passes a fake that appends the chime name to a list, then asserts the list is `["done"]` after a `CommandExecuted` is handled.